

GraphOne / FrontierAtlas — Ingestion Pipeline Architecture

AI Engineer Demo Task — Technical Architecture Document

1. Scale Strategy: Collecting 500,000+ Records Without Manual Intervention

The pipeline is split into three horizontally-independent stages -- fetch, extract, resolve -- connected by durable queues (not direct function calls), so each stage scales on its own axis and a slowdown in one never blocks the others.

- **Fetch (I/O-bound):** An **AdaptiveFetcher** (`src/scrapper/fetcher.py`) tries a cheap async HTTP request first (`aiohttp`) and only escalates to a real headless browser (Playwright) when the response looks bot-blocked (403/503, or a Cloudflare/Datadome challenge marker in the body). Because >90% of directory/listing pages are plain HTML, this keeps the median cost-per-page low, and horizontal scale is just 'run more worker processes/pods against more URL-queue partitions' -- no code changes, only replica count.
- **Extract (compute-bound, rate-limited by vendor):** The **LLMOrchestrator** (`src/llm/orchestrator.py`) processes chunks with bounded concurrency (`asyncio.Semaphore`) per worker; scaling to 500k records means running more worker replicas, each with its own concurrency budget, against a shared work queue -- again no code change. The 3-tier fallback (Gemini Flash -> Groq Llama 3 -> DeepSeek) means the effective extraction throughput is the SUM of all three providers' rate limits, not the minimum of one.
- **Resolve (CPU-bound, embarrassingly parallel):** Entity resolution is a pure function of (`raw_name`, `current canonical index`) with no external calls, so it scales linearly with CPU cores. The only shared-state concern is the canonical index itself when new entities are registered concurrently -- addressed via a single-writer queue (see Freshness Tracking below for the same underlying pattern).

Concretely: a work-queue-per-vertical (Startups, Products, Papers, Jobs, News) fed by seed-URL crawlers (directory pagination for startups/products, arXiv/HF listing APIs for papers, RSS/sitemap polling for news, board-specific pagination for jobs), consumed by a pool of fetch workers, whose output feeds an extraction queue consumed by a pool of LLM workers, whose output feeds a resolution+load queue. Each pool's replica count is an infrastructure dial, not a code change -- this is the literal mechanism behind 'scale to 500k with infra, not code.'

2. Handling 413s and 429s Across Thousands of Concurrent Extractions

413 Payload Too Large (proactive + reactive):

Proactively, `src/llm/chunker.py` splits any scraped text on paragraph/heading boundaries to stay under `max_tokens_per_chunk`, sized to the TIGHTEST provider in the fallback chain -- so a chunk that clears tier 1 is guaranteed to clear tiers 2 and 3 as well, making a 413-on-fallthrough essentially impossible. Reactively, if a 413 still occurs (e.g. a single oversized paragraph), the orchestrator halves the chunk and retries the SAME provider first (cheaper than switching tiers), recursing up to `max_chunk_splits` times before falling through.

429 Rate Limited (per-provider backoff, then fallthrough):

Each provider gets up to `max_retries_per_provider` attempts with exponential backoff plus jitter ($\text{base} * 2^{\text{attempt}} + \text{random jitter}$, capped at `max_backoff_seconds`) before the orchestrator falls through to the next tier in the chain. Backoff is scoped per-provider-per-chunk, not global, so one rate-limited provider does not stall unrelated chunks being processed by other workers. At fleet scale, each worker additionally tracks a rolling estimate of remaining quota per provider (from response headers where available) and proactively throttles its own concurrency before hitting 429s at all -- backoff-after-429 is the safety net, not the primary control.

3. Freshness Tracking Across Distributed Crawler Nodes

Deduplication is anchored on a canonical identity per vertical, not the crawl timestamp: (`source_domain`, `canonical_article_url`) for news, (`job_board`, `requisition_id`) for jobs, arXiv ID for papers. Every worker checks a shared, centrally-addressable key-value store (Redis, or a Postgres table with a UNIQUE constraint used as a de-dup gate) BEFORE enqueueing extraction work for a URL -- an atomic SETNX-style check-and-set means two crawler nodes

racing on the same URL only result in one of them proceeding, no matter how many nodes are running.

For date normalization: absolute dates in meta tags are parsed and stored in UTC ISO-8601 directly. Relative dates ('2 hours ago') are resolved against the HTTP response's Date header (not local wall-clock, which drifts across nodes) at fetch time. Where a source exposes neither, we apply the heuristic: a URL/requisition ID is 'new' if and only if it is absent from the de-dup store at check time -- meaning the FIRST time any node observes it, it is fresh by definition, and it never re-qualifies as fresh on a later crawl even if the page content changes, unless the content hash changes materially (re-published/updated article), which is tracked separately as an 'updated_at' rather than a new-record event.

4. Storage Strategy

Layer	Choice	Why
Primary record store	PostgreSQL	Strong schema + JSONB for the nested content.* fields gives us both relational integrity and flexibility
Queueing	Redis Streams / SQS	Durable work queues between fetch -> extract -> resolve stages; Redis also doubles as a cache
Relationship / graph layer	Neo4j (or Postgres + pg_graph_ext)	Extensible and scales well. Neo4j is a dedicated graph database. Postgres + pg_graph_ext is a general purpose database with a graph extension.
Vector layer	pgvector (colocated with Postgres)	Embeddings are stored in the same database as the records they point to. This simplifies search and duplication detection.

Rationale for splitting graph and vector layers rather than picking one: relationship traversal (graph) and semantic similarity (vector) answer fundamentally different questions, and forcing one engine to do both (e.g. a pure vector DB simulating relationships via metadata filters) degrades either the traversal performance or the recall quality. Postgres remains the system of record throughout -- both Neo4j and the vector store are kept in sync via change-data-capture from Postgres (Debezium or a simple outbox-table pattern), so there is a single source of truth and no risk of the graph/vector views drifting from the canonical data.

Companion engineering artifacts: src/llm/ (multi-tier extraction, tested), src/entity_resolution/ (canonicalization, tested), src/scrapper/ (adaptive fetcher + GitHub star tracking). See README.md for setup and the demo-scale data pull methodology.